



AG2 - Actividad Guiada 2

Nombre: Victor Vicente Pereira Torrealba

Link: [https://colab.research.google.com/drive/](https://colab.research.google.com/drive/1AsrUqs24XTad6QJ0Yr1n5BqikqYR2L9I)

[1AsrUqs24XTad6QJ0Yr1n5BqikqYR2L9I](https://colab.research.google.com/drive/1AsrUqs24XTad6QJ0Yr1n5BqikqYR2L9I)

Github: [https://github.com/vpereira3000/VIU-AI/blob/main/](https://github.com/vpereira3000/VIU-AI/blob/main/Algoritmos_Victor_Vicente_Pereira_Torrealba_AG2_actualizado.ipynb)

[Algoritmos_Victor_Vicente_Pereira_Torrealba_AG2_actualizado.ipynb](https://github.com/vpereira3000/VIU-AI/blob/main/Algoritmos_Victor_Vicente_Pereira_Torrealba_AG2_actualizado.ipynb)

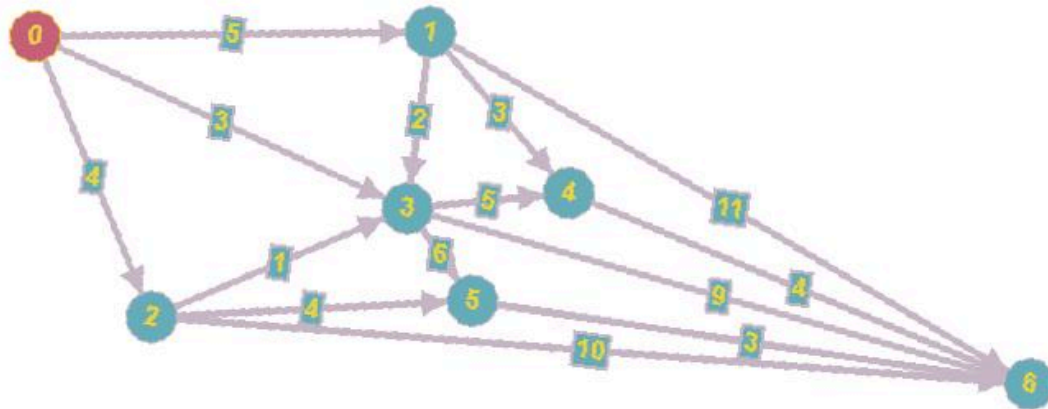
```
In [1]: import math
```

Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: “en una secuencia optima de decisiones, toda sub-secuencia también es óptima” (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla TARIFAS(i,j) para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
In [3]: #Viaje por el rio - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999], #desde nodo 0
[999,0,999,2,3,999,11], #desde nodo 1
[999,999, 0,1,999,4,10], #desde nodo 2
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS
```

```
Out[3]: [[0, 5, 4, 3, inf, 999, 999],
[999, 0, 999, 2, 3, 999, 11],
[999, 999, 0, 1, 999, 4, 10],
[999, 999, 999, 0, 5, 6, 9],
[999, 999, 999, 999, 0, 999, 4],
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
In [4]: #Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a otro
#####
def Precios(TARIFAS):
```

```
#####
#Total de Nodos
N = len(TARIFAS[0])

#Inicialización de la tabla de precios
PRECIOS = [ [9999]*N for i in [9999]*N]  #n x n
RUTA = [ [""]*N for i in [""]*N]

#Se recorren todos los nodos con dos bucles(origen - destino)
# para ir construyendo la matriz de PRECIOS
for i in range(N-1):
    for j in range(i+1, N):

        # Asumimos que el mínimo es la tarifa de un nodo a otro inicialmente
        MIN = TARIFAS[i][j]
        print("La tarifa desde " + str(i) + " hasta " + str(j) + " con un precio " + str(MIN))

        # Asumimos que la mejor ruta de un nodo a otro comienza en el nodo inicial
        RUTA[i][j] = i

        # Recorremos posibles movimientos intermedios
        for k in range(i, j):

            # Si el precio de ir de i a k más la tarifa de k a j es menor que la
            if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                RUTA[i][j] = k
                print("Ruta indirecta identificada desde " + str(i) + " hasta " + str(j) + " con un precio " + str(MIN))

        # Actualizamos precio de i a j como el mínimo (puede ser pasando por k)
        PRECIOS[i][j] = MIN

return PRECIOS,RUTA
```

```
In [5]: PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])
```

La tarifa desde 0 hasta 1 con un precio de 5
 La tarifa desde 0 hasta 2 con un precio de 4
 La tarifa desde 0 hasta 3 con un precio de 3
 La tarifa desde 0 hasta 4 con un precio de inf
 Ruta indirecta identificada desde 0 hasta 4 pasando por 1 con un precio de 8
 La tarifa desde 0 hasta 5 con un precio de 999
 Ruta indirecta identificada desde 0 hasta 5 pasando por 2 con un precio de 8
 La tarifa desde 0 hasta 6 con un precio de 999
 Ruta indirecta identificada desde 0 hasta 6 pasando por 1 con un precio de 16
 Ruta indirecta identificada desde 0 hasta 6 pasando por 2 con un precio de 14
 Ruta indirecta identificada desde 0 hasta 6 pasando por 3 con un precio de 12
 Ruta indirecta identificada desde 0 hasta 6 pasando por 5 con un precio de 11
 La tarifa desde 1 hasta 2 con un precio de 999
 La tarifa desde 1 hasta 3 con un precio de 2
 La tarifa desde 1 hasta 4 con un precio de 3
 La tarifa desde 1 hasta 5 con un precio de 999
 Ruta indirecta identificada desde 1 hasta 5 pasando por 3 con un precio de 8
 La tarifa desde 1 hasta 6 con un precio de 11
 Ruta indirecta identificada desde 1 hasta 6 pasando por 4 con un precio de 7
 La tarifa desde 2 hasta 3 con un precio de 1
 La tarifa desde 2 hasta 4 con un precio de 999
 Ruta indirecta identificada desde 2 hasta 4 pasando por 3 con un precio de 6
 La tarifa desde 2 hasta 5 con un precio de 4
 La tarifa desde 2 hasta 6 con un precio de 10
 Ruta indirecta identificada desde 2 hasta 6 pasando por 5 con un precio de 7
 La tarifa desde 3 hasta 4 con un precio de 5
 La tarifa desde 3 hasta 5 con un precio de 6
 La tarifa desde 3 hasta 6 con un precio de 9
 La tarifa desde 4 hasta 5 con un precio de 999
 La tarifa desde 4 hasta 6 con un precio de 4
 La tarifa desde 5 hasta 6 con un precio de 3

PRECIOS

```

[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
  
```

RUTA

```

['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
  
```

In [6]: *#Calculo de la ruta usando la matriz RUTA*

```

def calcular_ruta(RUTA, desde, hasta):
    if desde == RUTA[desde][hasta]:
        #if desde == hasta:
            #print("Ir a :" + str(desde))
  
```

```

        return desde
    else:
        return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ',' + str(RUTA[desde][hasta])

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

La ruta es:

Out[6]: '0,2,5'

Problema de Asignacion de tarea

```

In [7]: #Asignacion de tareas - Ramificación y Poda
#####
#      T A R E A
#      A
#      G
#      E
#      N
#      T
#      E

COSTES=[[11,12,18,40],
        [14,15,13,22],
        [11,17,19,23],
        [17,14,20,28]]

```

```

In [8]: #Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        VALOR += COSTES[S[i]][i]
    return VALOR

valor((3,2, ),COSTES)

```

Out[8]: 34

```

In [9]: #Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

def CI(S,COSTES):
    # Calculo el coste parcial de la solución dada
    coste = valor(S,COSTES)

    # Calculo coste mínimo de opciones restantes no usadas permitiendo reutili
    no_usadas = set(range(len(COSTES)))-set(S)

```

```

    for i in range(len(S),len(COSTES)):
        coste += min([COSTES[i][j] for j in no_usadas])

    return coste

def CS(S,COSTES):
    # Calculo el coste parcial de la solución dada
    coste = valor(S,COSTES)

    # Calculo coste máximo de opciones restantes no usadas
    no_usadas = set(range(len(COSTES)))-set(S)

    for i in range(len(S),len(COSTES)):
        coste += max([COSTES[i][j] for j in no_usadas])

    return coste

CI((0,1),COSTES)

```

Out[9]: 65

```

In [10]: #Genera tantos hijos como posibilidades haya para la siguiente elemento c
#(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N ):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS

```

```

In [11]: crear_hijos((0,) , 4)

```

Out[11]: [{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

```

In [12]: def ramificacion_y_poda(COSTES):
#Construccion iterativa de soluciones(arbol). En cada etapa asignamos un agent
#Nodos del grafo { s:(1,2),CI:3,CS:5 }
print(COSTES)
DIMENSION = len(COSTES)
MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
CotaSup = valor(MEJOR_SOLUCION,COSTES)
print("Cota Superior:", CotaSup)

NODOS=[]
NODOS.append({'s':(), 'ci':CI((),COSTES) })

iteracion = 0

while( len(NODOS) > 0):
    iteracion +=1

    nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
    print("Nodo prometedor:", nodo_prometedor)

```

```

#Ramificacion
#Se generan los hijos
HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in crear_hijos(nodo) ]
print("Explorando hijos:")
print(HIJOS)

#Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos
NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
if len(NODO_FINAL) > 0:
    print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMENSION ])
    if NODO_FINAL[0]['ci'] < CotaSup:
        CotaSup = NODO_FINAL[0]['ci']
        MEJOR_SOLUCION = NODO_FINAL

#Poda
HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]
print("Despues de la poda:")
print(HIJOS)

#Añadimos los hijos
NODOS.extend(HIJOS)

#Eliminamos el nodo ramificado
NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]
print("NODOS a explorar: ")
print(NODOS)

print("La solucion final es:" ,MEJOR_SOLUCION , " en " , iteracion , " iteraciones")

def ramificacion_y_poda(COSTES)

```

```

[[11, 12, 18, 40], [14, 15, 13, 22], [11, 17, 19, 23], [17, 14, 20, 28]]
Cota Superior: 73
Nodo prometededor: ()
Explorando hijos:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
Despues de la poda:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
Nodo prometededor: (2,)
Explorando hijos:
[{'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
Despues de la poda:
[{'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
Nodo prometededor: (2, 3)
Explorando hijos:
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
Despues de la poda:
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
Nodo prometededor: (2, 0)
Explorando hijos:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
Despues de la poda:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
Nodo prometededor: (2, 1)
Explorando hijos:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
Despues de la poda:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
Nodo prometededor: (0,)
Explorando hijos:
[{'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]
Despues de la poda:
[{'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]
NODOS a explorar:
[{'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57},

```


{'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]

Nodo prometedor: (1,)

Explorando hijos:

[{'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]

Despues de la poda:

[{'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]

NODOS a explorar:

[{'s': (3,), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]

Nodo prometedor: (3,)

Explorando hijos:

[{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]

Despues de la poda:

[{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]

NODOS a explorar:

[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]

Nodo prometedor: (2, 3, 1)

Explorando hijos:

[{'s': (2, 3, 1, 0), 'ci': 78}]

*****Soluciones: [{'s': (2, 3, 1, 0), 'ci': 78}]

Despues de la poda:

[]

NODOS a explorar:

[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]

Nodo prometedor: (0, 3)

Explorando hijos:

[{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]

Despues de la poda:

[{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]

NODOS a explorar:

[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]

Nodo prometedor: (1, 3)

Explorando hijos:

[{'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]

Despues de la poda:

```
[{'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
NODOS a explorar:
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (2, 3, 0)
Explorando hijos:
[{'s': (2, 3, 0, 1), 'ci': 65}]
```

*****Soluciones: [{'s': (2, 3, 0, 1), 'ci': 65}]

Despues de la poda:

[]

NODOS a explorar:

```
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (2, 0, 3)

Explorando hijos:

```
[{'s': (2, 0, 3, 1), 'ci': 65}]
```

*****Soluciones: [{'s': (2, 0, 3, 1), 'ci': 65}]

Despues de la poda:

[]

NODOS a explorar:

```
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (0, 3, 1)

Explorando hijos:

```
[{'s': (0, 3, 1, 2), 'ci': 61}]
```

*****Soluciones: [{'s': (0, 3, 1, 2), 'ci': 61}]

Despues de la poda:

[]

NODOS a explorar:

```
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (0, 3, 2)

Explorando hijos:

```
[{'s': (0, 3, 2, 1), 'ci': 66}]
```

*****Soluciones: [{'s': (0, 3, 2, 1), 'ci': 66}]

Despues de la poda:

```

[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (0, 2)
Explorando hijos:
[{'s': (0, 2, 1), 'ci': 69}, {'s': (0, 2, 3), 'ci': 62}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (1, 2)
Explorando hijos:
[{'s': (1, 2, 0), 'ci': 77}, {'s': (1, 2, 3), 'ci': 68}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 2)
Explorando hijos:
[{'s': (3, 2, 0), 'ci': 66}, {'s': (3, 2, 1), 'ci': 64}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 0)
Explorando hijos:
[{'s': (3, 0, 1), 'ci': 62}, {'s': (3, 0, 2), 'ci': 62}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 1), 'ci': 60}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 1)
Explorando hijos:
[{'s': (3, 1, 0), 'ci': 70}, {'s': (3, 1, 2), 'ci': 68}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]

```

```
6}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (2, 1, 3)
Explorando hijos:
[{'s': (2, 1, 3, 0), 'ci': 86}]
```

```
*****Soluciones: [{'s': (2, 1, 3, 0), 'ci': 86}]
```

```
Despues de la poda:
```

```
[]
```

```
NODOS a explorar:
```

```
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

```
Nodo prometedor: (2, 0, 1)
```

```
Explorando hijos:
```

```
[{'s': (2, 0, 1, 3), 'ci': 64}]
```

```
*****Soluciones: [{'s': (2, 0, 1, 3), 'ci': 64}]
```

```
Despues de la poda:
```

```
[]
```

```
NODOS a explorar:
```

```
[{'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

```
Nodo prometedor: (1, 3, 2)
```

```
Explorando hijos:
```

```
[{'s': (1, 3, 2, 0), 'ci': 87}]
```

```
*****Soluciones: [{'s': (1, 3, 2, 0), 'ci': 87}]
```

```
Despues de la poda:
```

```
[]
```

```
NODOS a explorar:
```

```
[{'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}]
```

```
Nodo prometedor: (0, 1)
```

```
Explorando hijos:
```

```
[{'s': (0, 1, 2), 'ci': 73}, {'s': (0, 1, 3), 'ci': 66}]
```

```
Despues de la poda:
```

```
[]
```

```
NODOS a explorar:
```

```
[{'s': (2, 1, 0), 'ci': 72}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}]
```

```
Nodo prometedor: (1, 0)
```

```
Explorando hijos:
```

```
[{'s': (1, 0, 2), 'ci': 73}, {'s': (1, 0, 3), 'ci': 66}]
```

```
Despues de la poda:
```

```
[]
```

```
NODOS a explorar:
```

```
[{'s': (2, 1, 0), 'ci': 72}, {'s': (1, 3, 0), 'ci': 66}]
```

```
Nodo prometedor: (1, 3, 0)
```

```
Explorando hijos:
```

```
[{'s': (1, 3, 0, 2), 'ci': 69}]
```

```
*****Soluciones: [{'s': (1, 3, 0, 2), 'ci': 69}]
```

```
Despues de la poda:
```

```
[]
```

```

NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}]
Nodo prometedor: (2, 1, 0)
Explorando hijos:
[{'s': (2, 1, 0, 3), 'ci': 72}]

*****Soluciones: [{'s': (2, 1, 0, 3), 'ci': 72}]
Despues de la poda:
[]
NODOS a explorar:
[]
La solucion final es: [{'s': (0, 3, 1, 2), 'ci': 61}] en 27 iteraciones par
a dimension: 4

```

Descenso del gradiente

```

In [13]: import math                                #Funciones matematicas
import matplotlib.pyplot as plt                    #Generacion de gráficos (otra opcion seaborn)
import numpy as np                                #Tratamiento matriz N-dimensionales y otras (
import scipy as sc

import random

```

Vamos a buscar el minimo de la funcion paraboloide : $f(x) = x^2 + y^2$

Obviamente se encuentra en (x,y)=(0,0) pero probaremos como llegamos a él a través del descenso del gradiente.

```

In [14]: #Definimos la funcion
#Paraboloide
f = lambda X: X[0]**2 + X[1]**2                    #Funcion
df = lambda X: [2*X[0], 2*X[1]]                   #Gradiente

df([1,2])

```

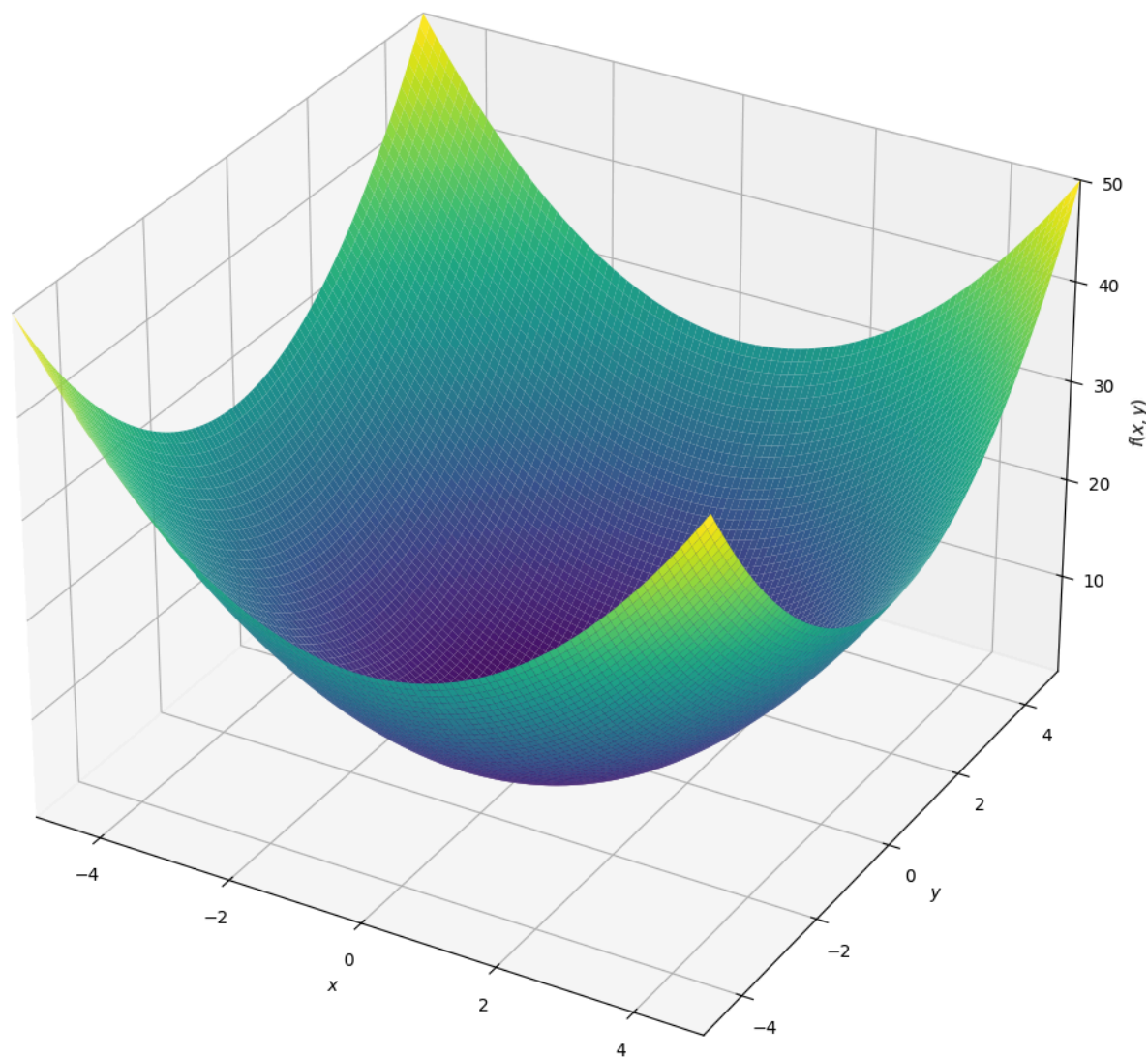
Out[14]: [2, 4]

```

In [15]: from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
plot3d(x**2 + y**2,
       (x,-5,5),(y,-5,5),
       title='x**2 + y**2',
       size=(10,10))

```

$$x^2 + y^2$$



Out[15]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x7e20fd56a570>

```
In [16]: #Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=5.5
```

```
X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])
```

```
#Pinta el mapa de niveles de Z
```

```

plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

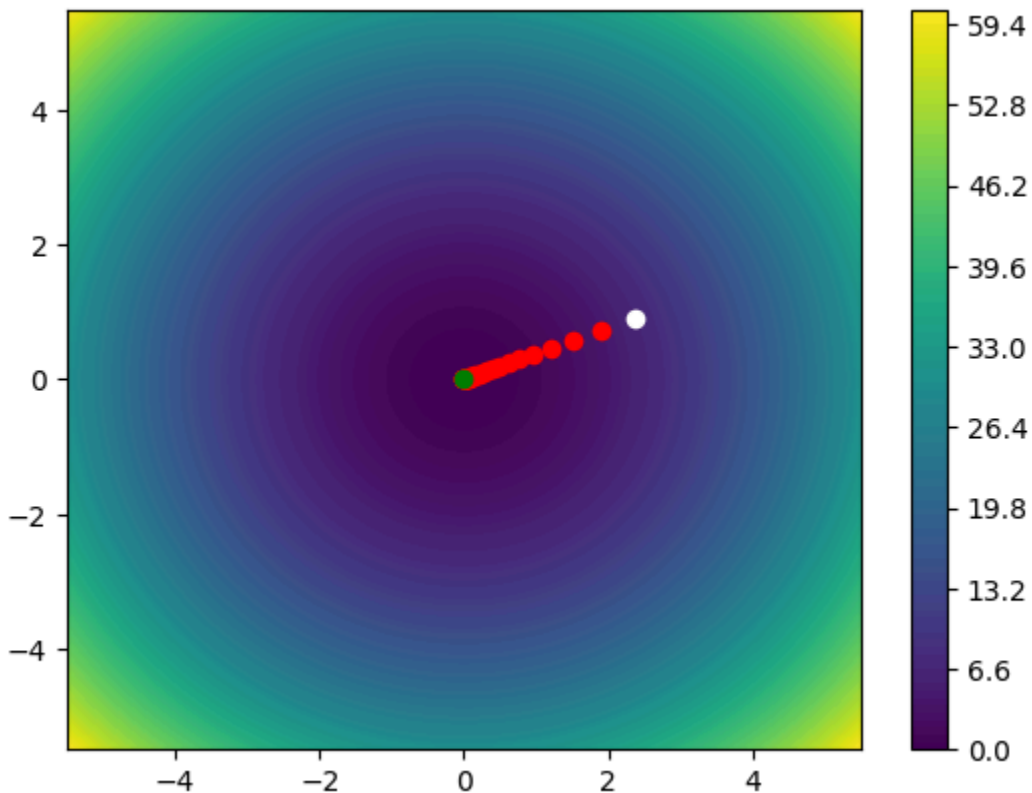
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-5,5 ),random.uniform(-5,5 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acer
TA=.1

#Iteraciones:50
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))

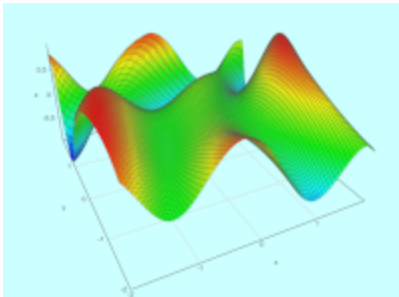
```



Solucion: [3.370852171707748e-05, 1.3086079963389312e-05] 1.3075099251589034e-09

¿Te atreves a optimizar la función?:

$f(x) = \sin\left(\frac{1}{2} * x^2 - \frac{1}{4} * y^2 + 3\right) * \cos(2*x + 1 - e^y)$



```
In [17]: #Definimos la funcion
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1

In [18]: #La función tiene muchos máximos y mínimos locales debido a los términos seno
#Por eso conviene usar un algoritmo de optimización global como differential_e
import math
from scipy.optimize import differential_evolution

# Función objetivo
def f(X):
    x, y = X

    return (
        math.sin(0.5*x**2 - 0.25*y**2 + 3)
        * math.cos(2*x + 1 - math.exp(y))
    )

# Intervalos de búsqueda para x e y
limites = [
    (-5, 5),    # límites de x
    (-5, 5)     # límites de y
]

# -----
# Encontrar el máximo de f
# -----
# scipy minimiza funciones.
# Para maximizar f, minimizamos -f.
resultado_max = differential_evolution(
    lambda X: -f(X),
    bounds=limites,
    seed=123,
    tol=1e-10,
    polish=True
)

x_max, y_max = resultado_max.x
valor_max = f(resultado_max.x)

# -----
```



```

# Encontrar el mínimo de f
# -----
resultado_min = differential_evolution(
    f,
    bounds=limites,
    seed=123,
    tol=1e-10,
    polish=True
)

x_min, y_min = resultado_min.x
valor_min = f(resultado_min.x)

print("Máximo")
print(f"x = {x_max:.8f}")
print(f"y = {y_max:.8f}")
print(f"f(x,y) = {valor_max:.12f}")

print("\nMínimo")
print(f"x = {x_min:.8f}")
print(f"y = {y_min:.8f}")
print(f"f(x,y) = {valor_min:.12f}")

```

Máximo

x = -3.11596041

y = 0.04999270

f(x,y) = 0.999999999998

Mínimo

x = -3.63560908

y = -4.42557828

f(x,y) = -0.999999999998

```

In [19]: #Los puntos encontrados por el programa no son únicos. Debido a la periodicidad
#existen múltiples valores de x e y donde la función alcanza 1 o -1.
#También se puede conservar exactamente la función lambda:
import math
from scipy.optimize import differential_evolution

f = lambda X: (
    math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3)
    * math.cos(2*X[0] + 1 - math.exp(X[1]))
)

limites = [(-5, 5), (-5, 5)]

resultado = differential_evolution(
    lambda X: -f(X),
    limites,
    seed=123
)

print("Punto óptimo:", resultado.x)

```

```
print("Valor máximo:", f(resultado.x))
```

Punto óptimo: [-3.11596081 0.049994]

Valor máximo: 0.9999999999992072